



Machine Vision with Machine Learning #2

FRA 626 Machine Vision in Smart Factory

Suriya Natsupakpong, PhD

Institute of Field Robotics (FIBO)

King Mongkut's University of Technology Thonburi (KMUTT)

Topics

- A Survey of the Recent Architectures of Deep Convolutional Neural Networks
- Principles of Training Multi-layer Neural Network using Backpropagation
- Convolutional Neural Networks : LeNet
- ML.NET

A Survey of the Recent Architectures of Deep Convolutional Neural Networks

- Application areas of CNN include Image Classification and Segmentation, Object Detection, Video Processing, Natural Language Processing, and Speech Recognition.
- The attractive feature of CNN is its ability to exploit spatial or temporal correlation in data.
- CNN learns through backpropagation algorithm, by regulating the change in weights according to the target.

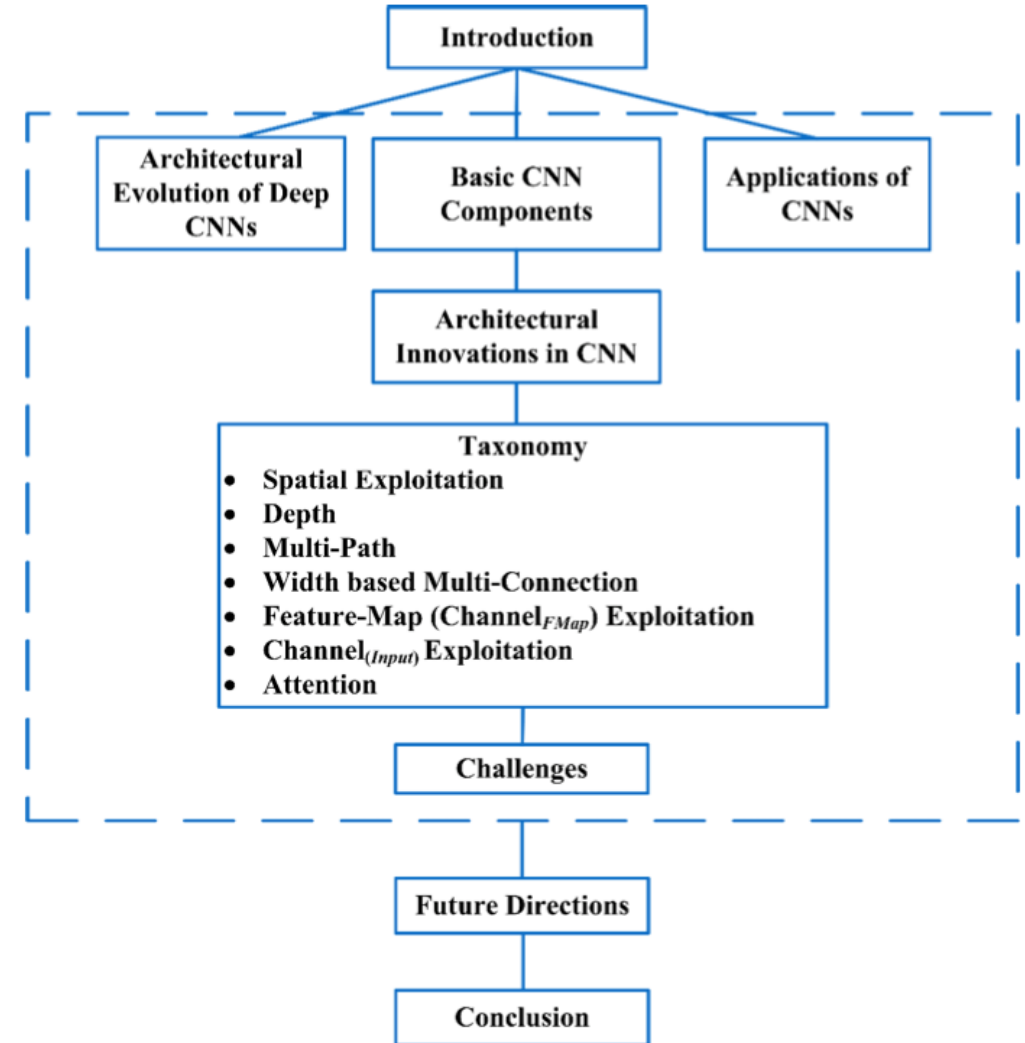
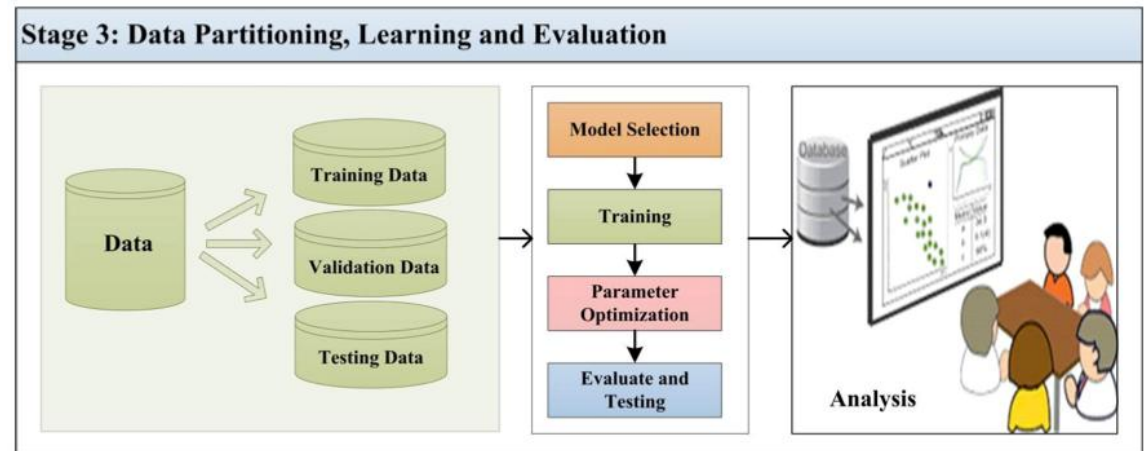
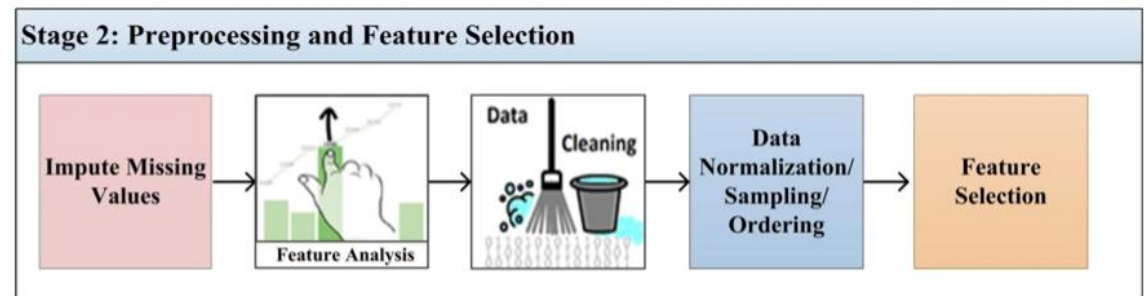
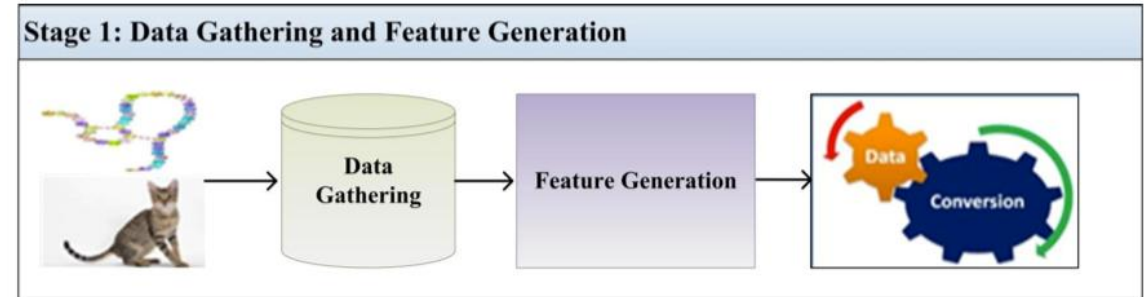


Fig. 1 Organization of the survey paper showing different sections.

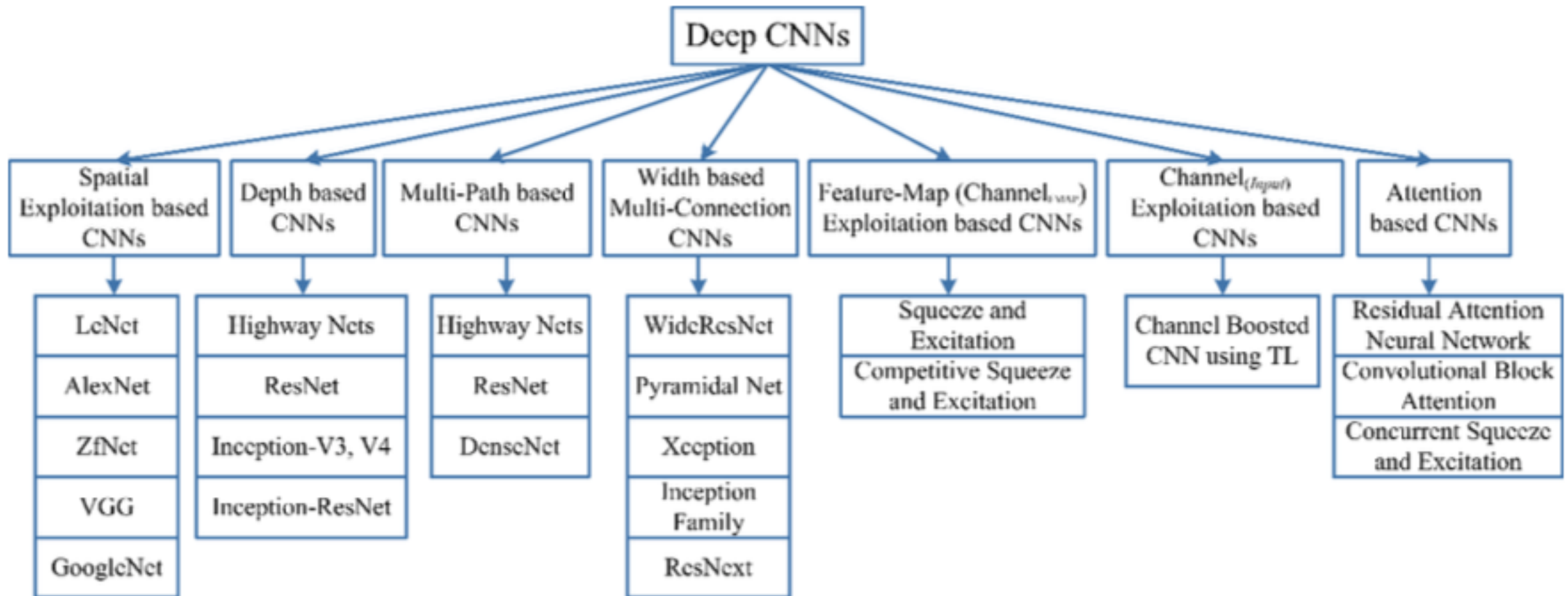
A Survey of the Recent Architectures of Deep Convolutional Neural Networks

Basic layout of a typical ML system having several stages.

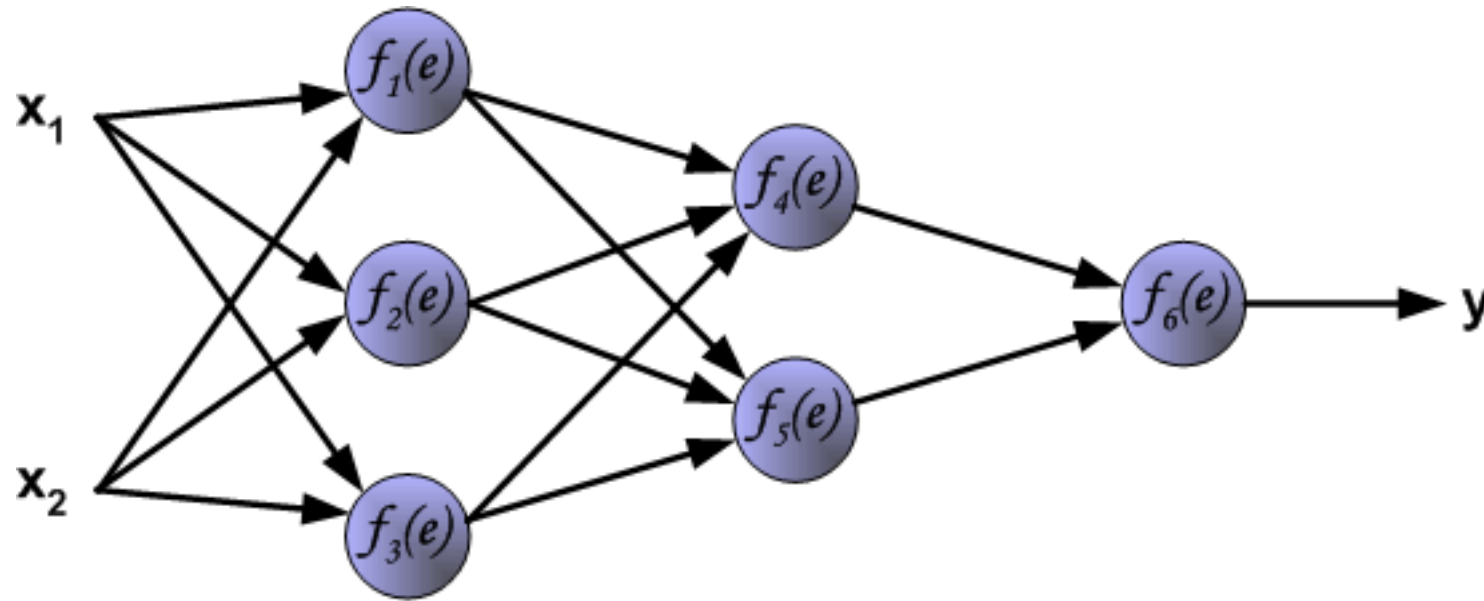


A Survey of the Recent Architectures of Deep Convolutional Neural Networks

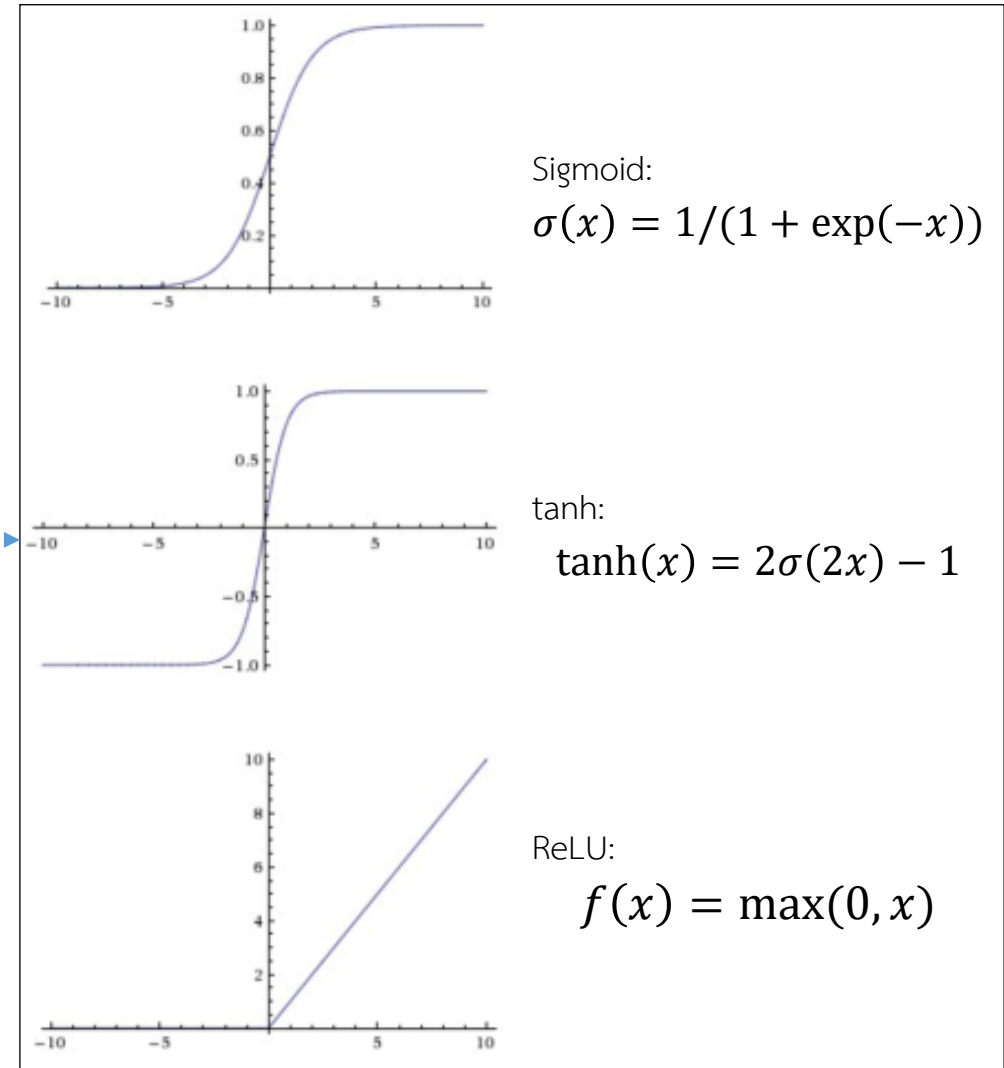
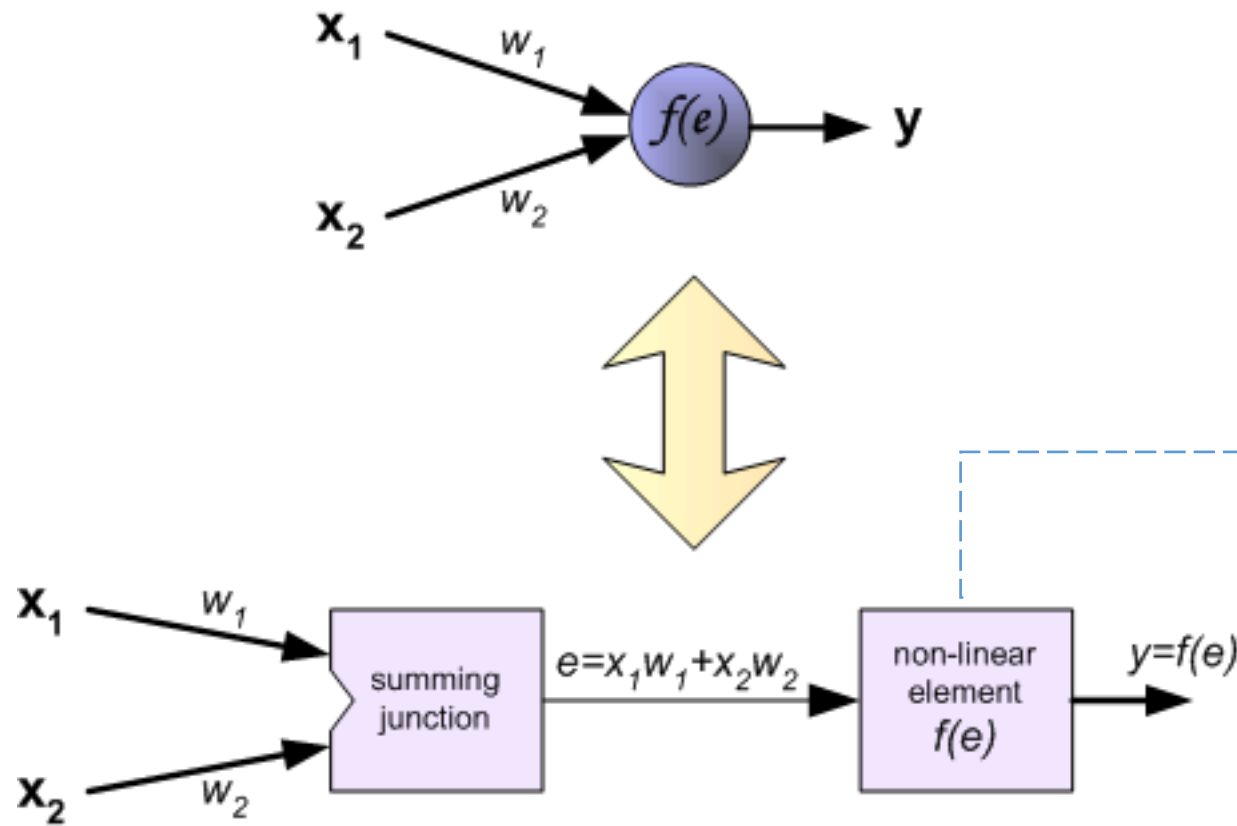
Taxonomy of deep CNN architectures showing seven different categories.



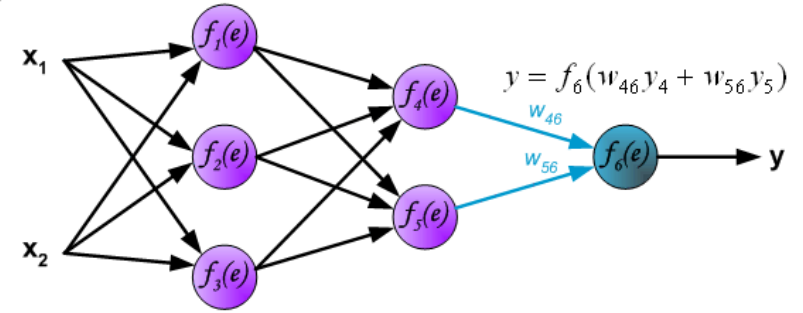
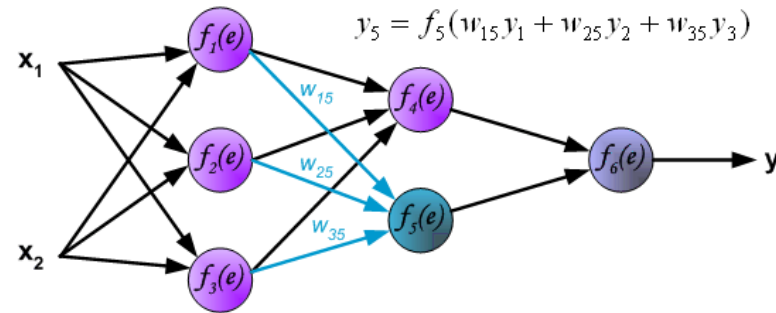
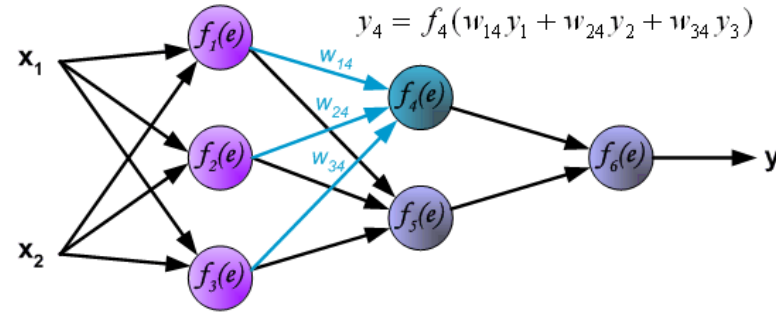
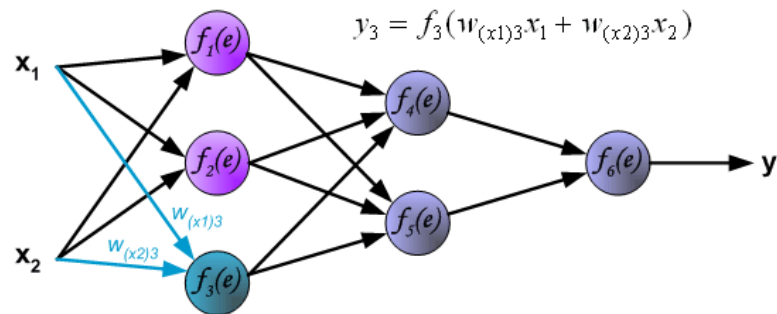
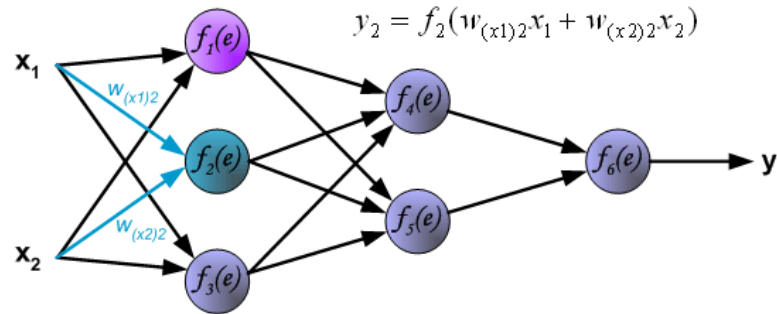
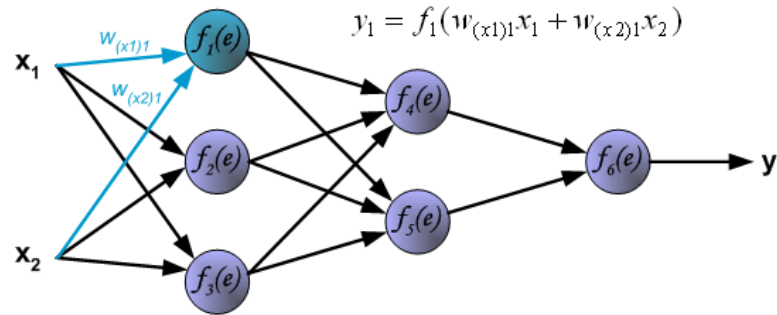
Principles of Training Multi-layer Neural Network using Backpropagation



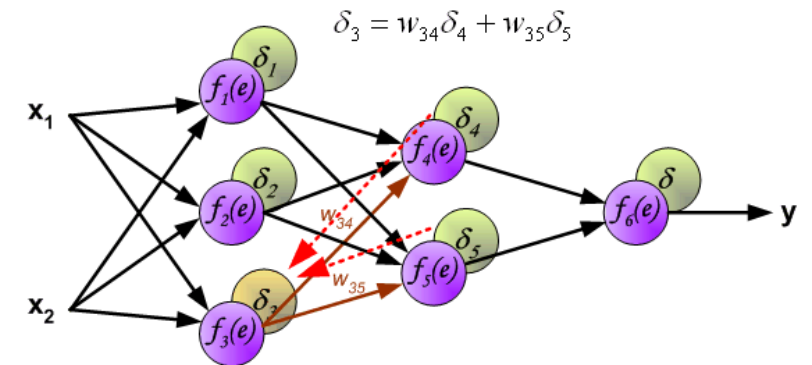
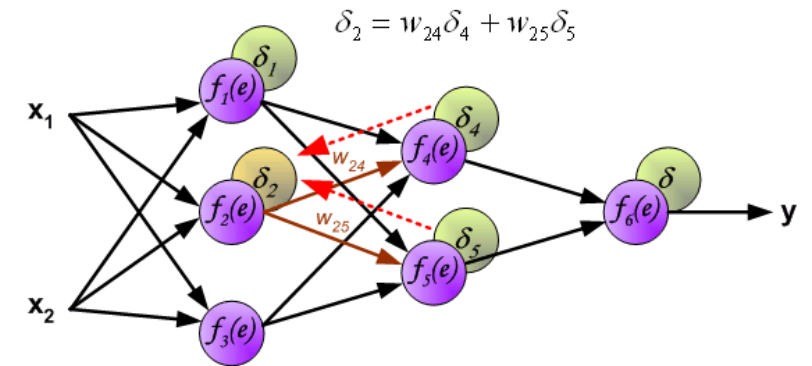
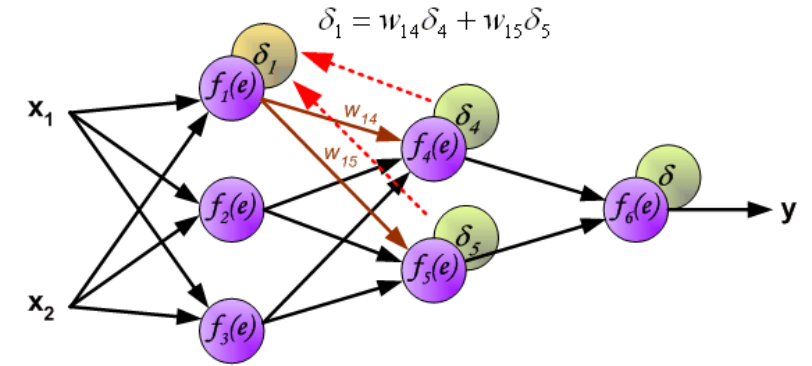
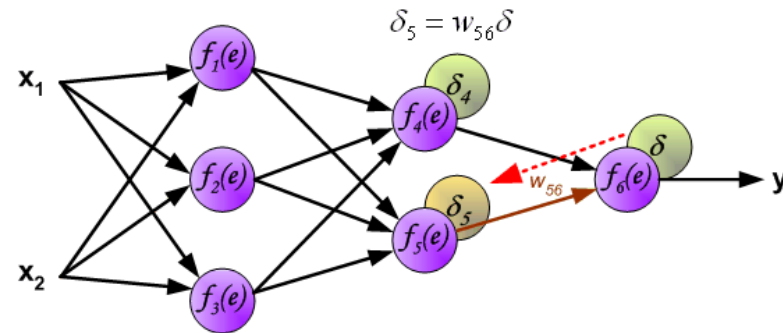
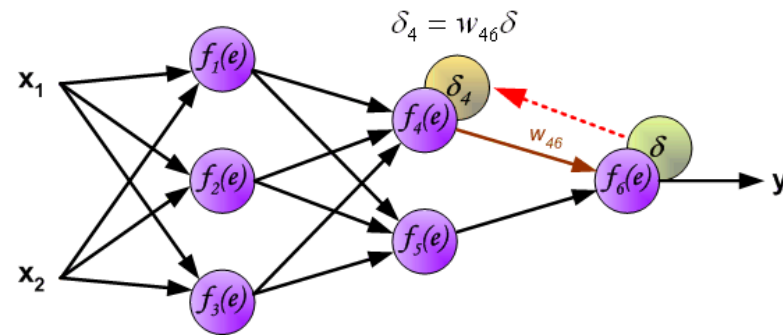
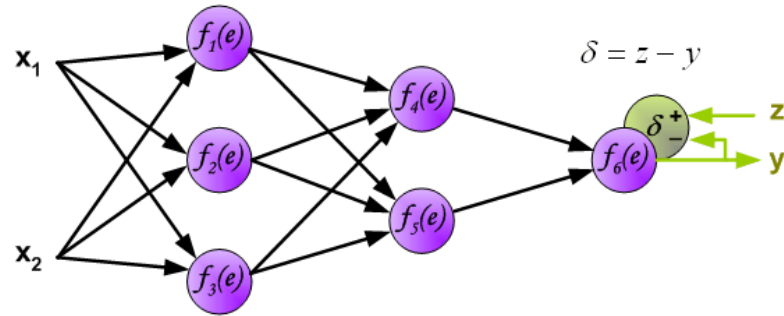
Principles of Training Multi-layer Neural Network using Backpropagation

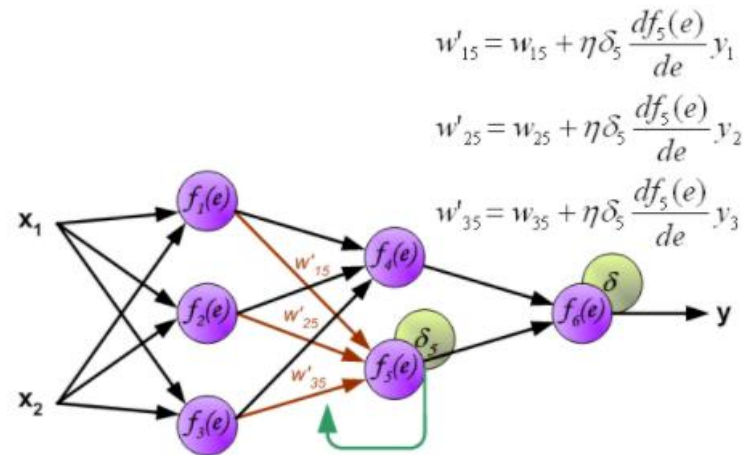
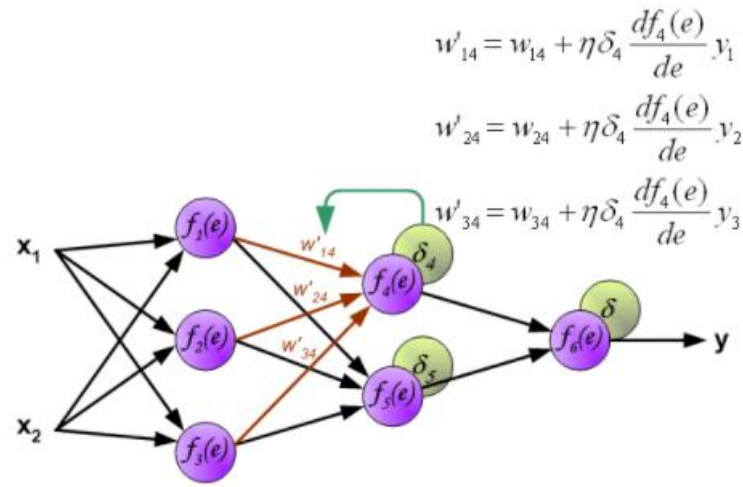
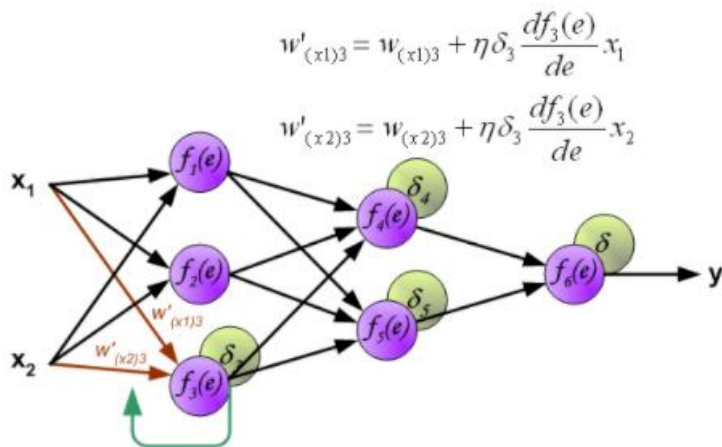
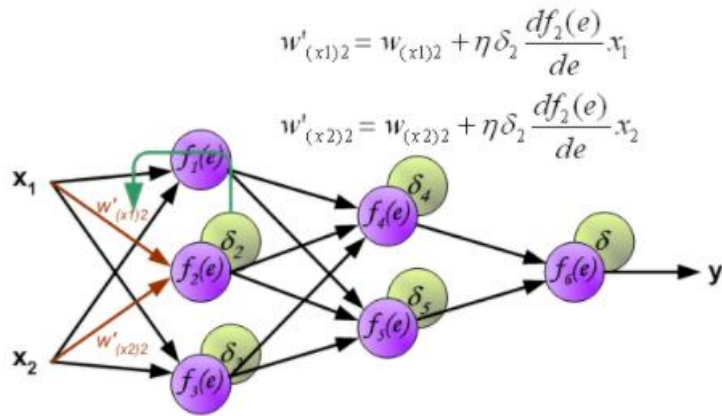
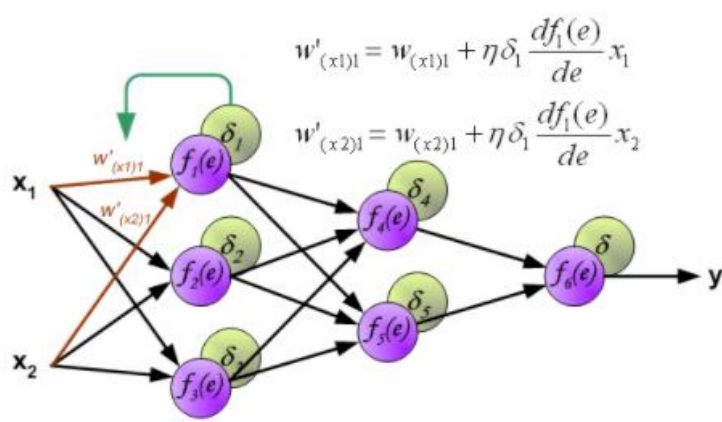


Forward Propagation Step

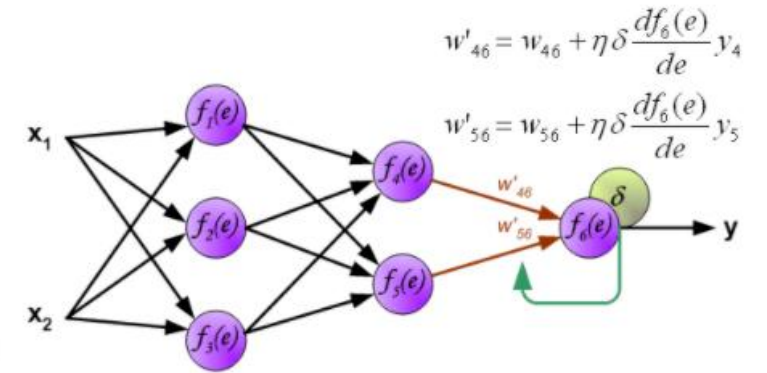


Backward Propagation Step





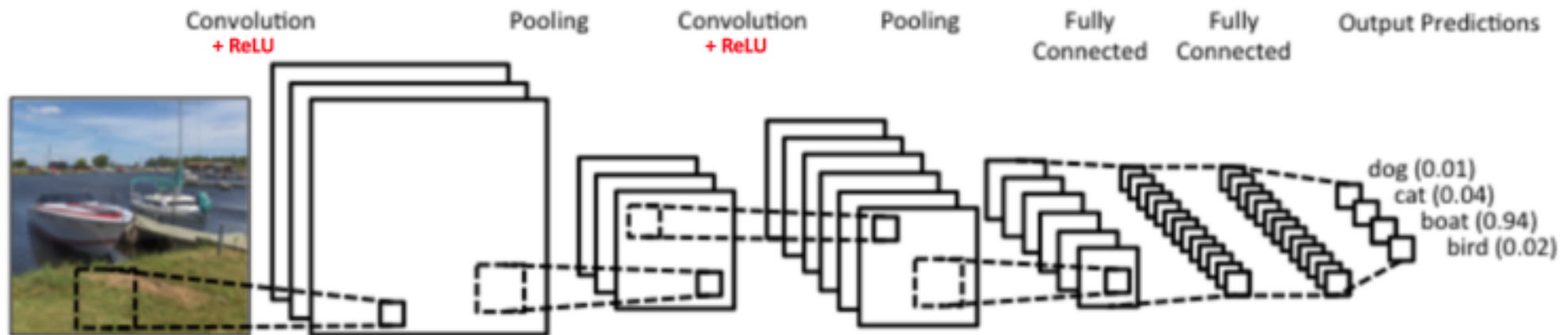
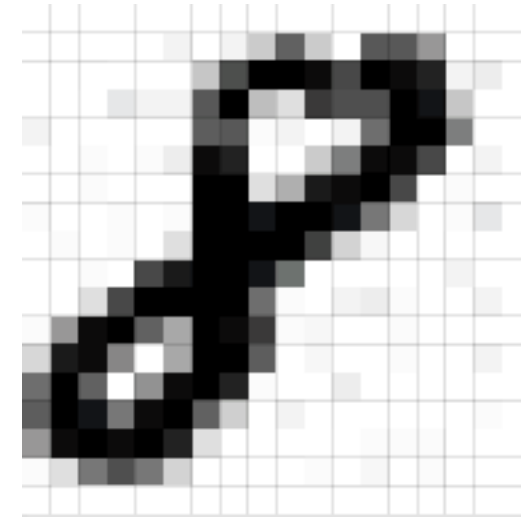
Weight Updating Step



LeNet

There are four main operations in the ConvNet

1. Convolution
2. Non Linearity (ReLU)
3. Pooling or Sub Sampling
4. Classification (Fully Connected Layer)



LeNet : Convolution Step

- ConvNet is to extract features from the input image.
- Convolution preserves the spatial relationship between pixels by learning image features using small squares of input data.

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

1	0	1
0	1	0
1	0	1

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

LeNet : Convolution Step

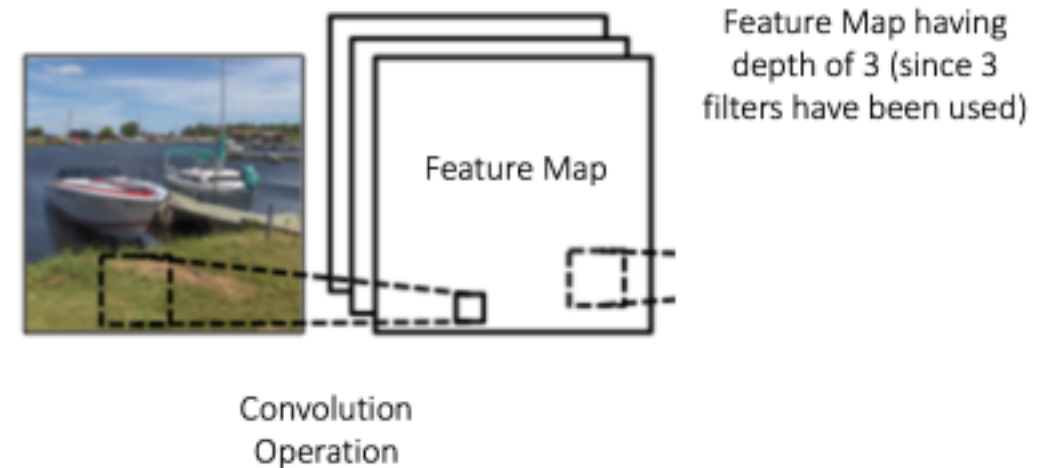
- a CNN learns the values of these filters on its own during the training process (although we still need to specify parameters such as number of filters, filter size, architecture of the network etc. before the training process).
- The more number of filters we have, the more image features get extracted and the better our network becomes at recognizing patterns in unseen images.



LeNet : Convolution Step

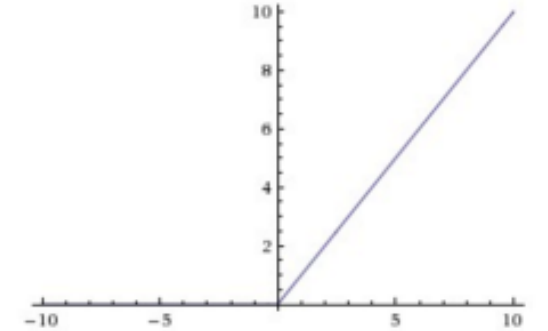
The size of the Feature Map (Convolved Feature) is controlled by three parameters.

- **Depth:** Depth corresponds to the number of filters we use for the convolution operation.
- **Stride:** Stride is the number of pixels by which we slide our filter matrix over the input matrix. When the stride is 1 then we move the filters one pixel at a time.
- **Zero-padding:** Sometimes, it is convenient to pad the input matrix with zeros around the border, so that we can apply the filter to bordering elements of our input image matrix.

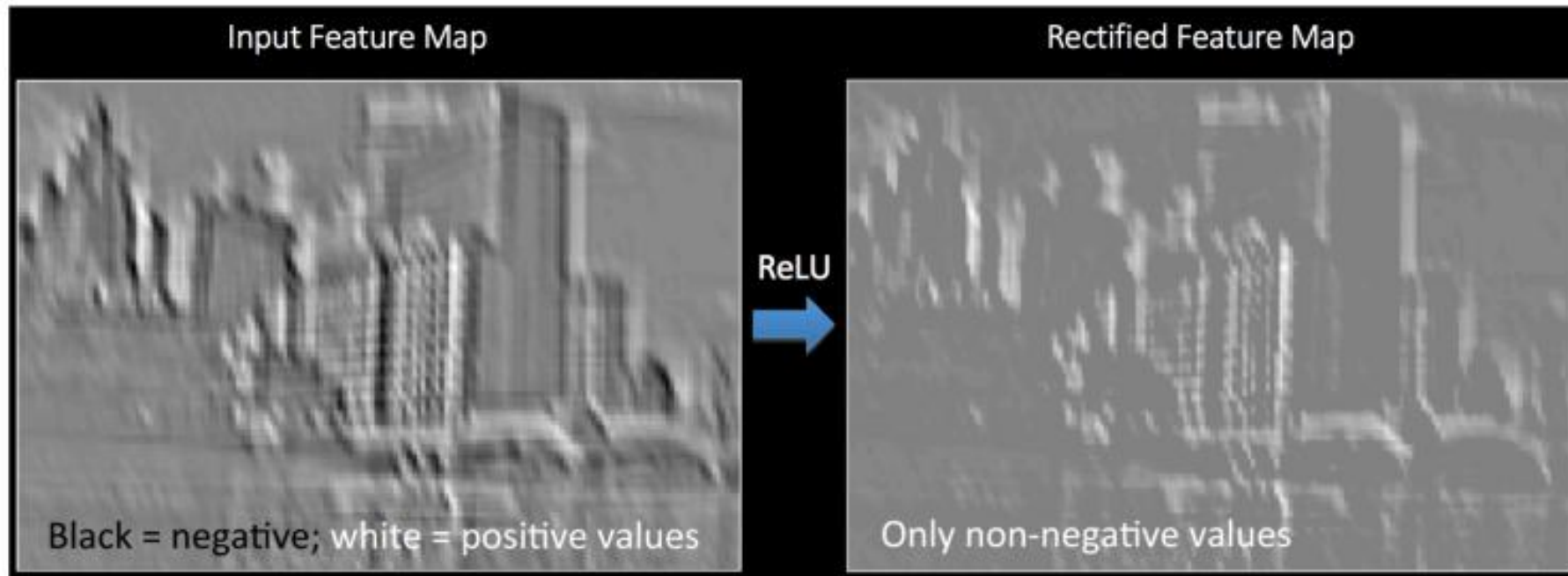


LeNet : Non Linearity (ReLU)

$$\text{Output} = \text{Max}(\text{zero}, \text{Input})$$

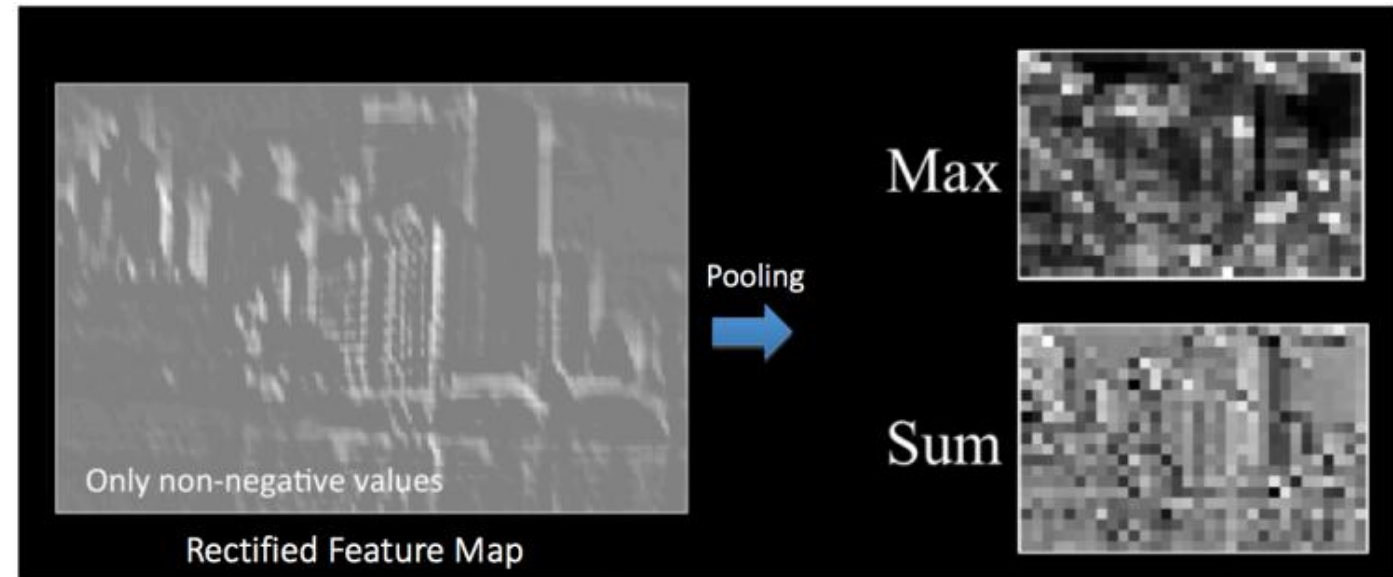
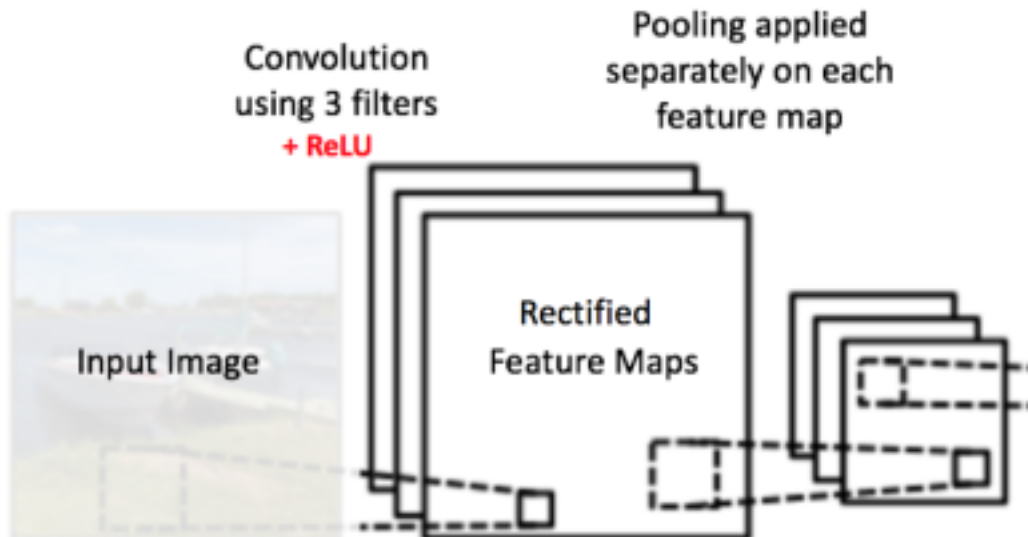
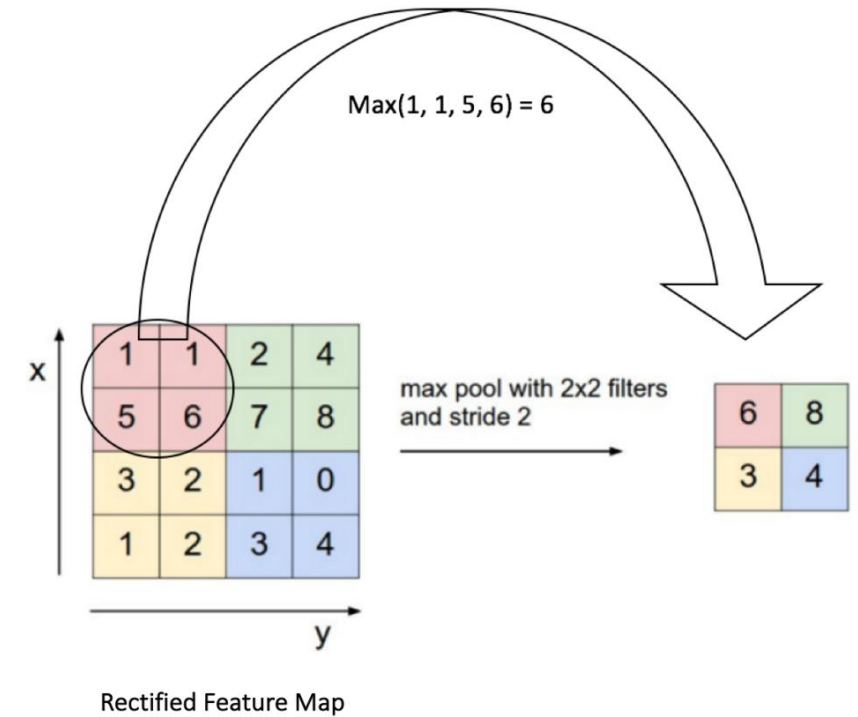


- ReLU stands for Rectified Linear Unit and is a non-linear operation.
- ReLU has been used after every Convolution operation.
- ReLU is an element wise operation (applied per pixel) and replaces all negative pixel values in the feature map by zero.



LeNet : Pooling or Sub Sampling

- Spatial Pooling (also called subsampling or downsampling) reduces the dimensionality of each feature map but retains the most important information.
- Spatial Pooling can be of different types: Max, Average, Sum etc.
- Pooling operation is applied separately to each feature map.

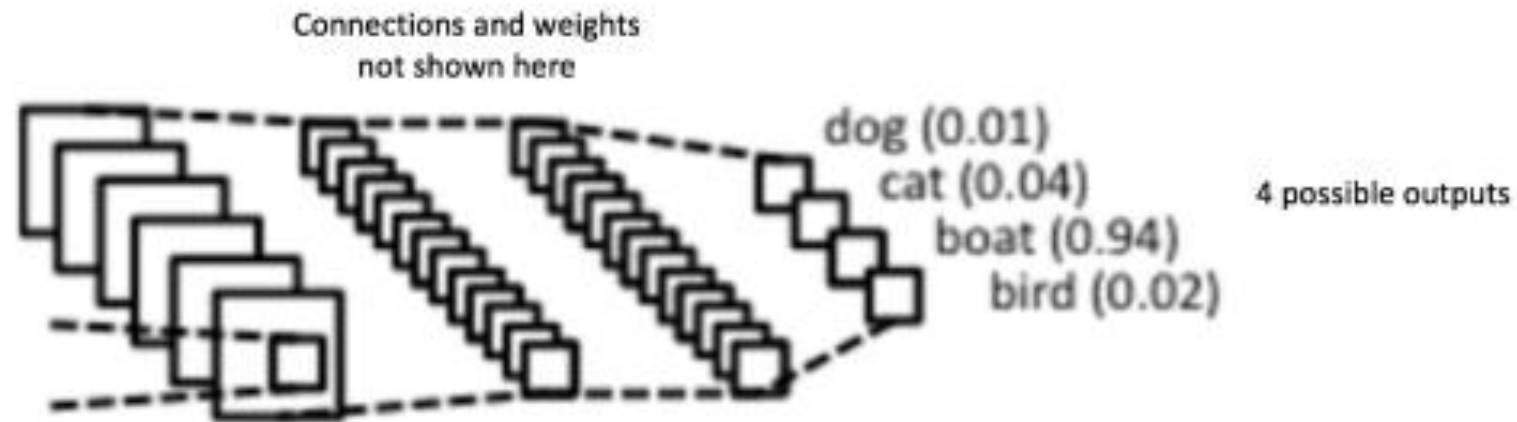


LeNet : Pooling or Sub Sampling

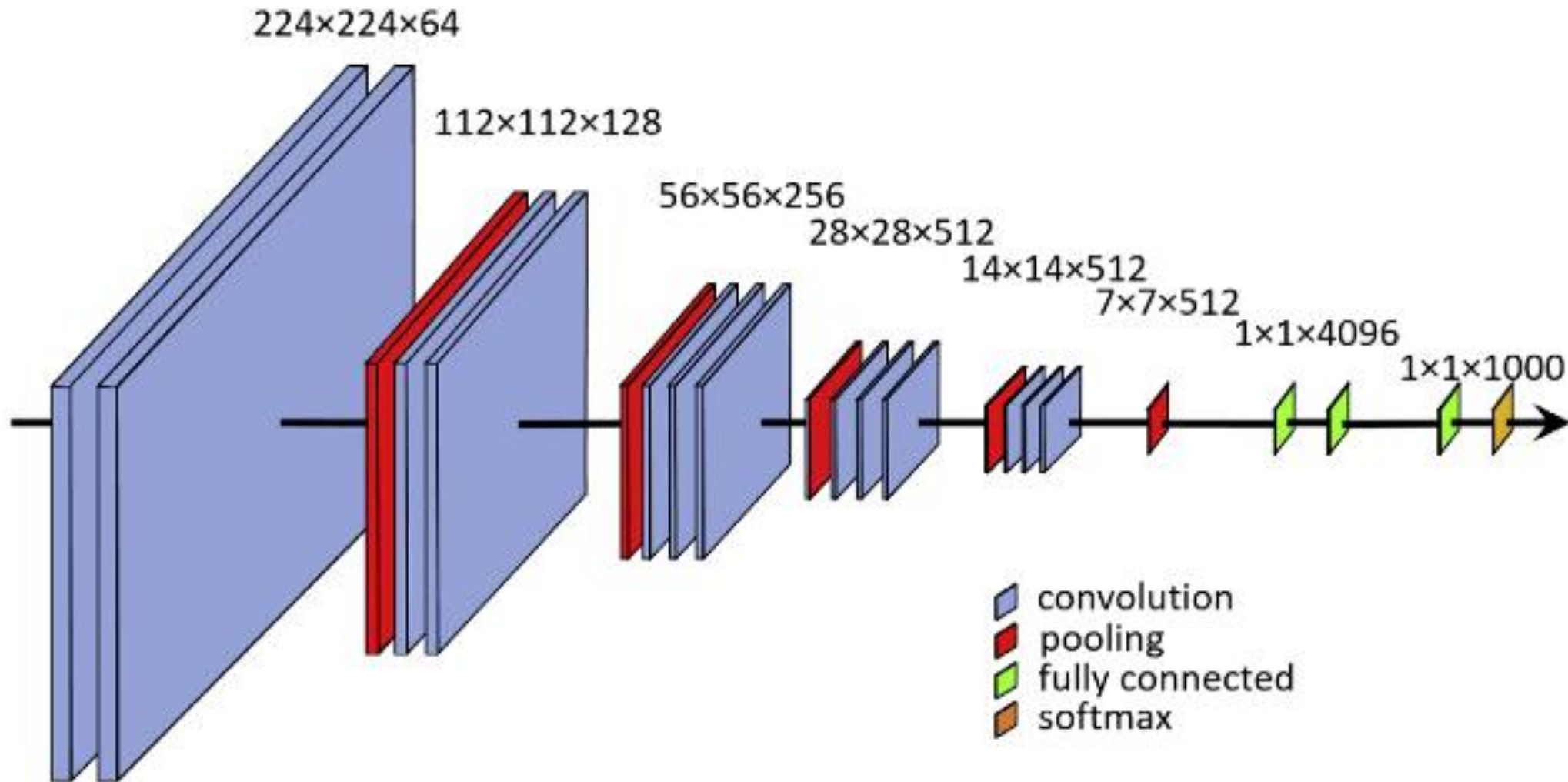
- Makes the input representations (feature dimension) smaller and more manageable
- Reduces the number of parameters and computations in the network, therefore, controlling overfitting.
- Makes the network invariant to small transformations, distortions and translations in the input image (a small distortion in input will not change the output of Pooling – since we take the maximum / average value in a local neighborhood).
- Helps us arrive at an almost scale invariant representation of our image (the exact term is “equivariant”). This is very powerful since we can detect objects in an image no matter where they are located.

LeNet : Classification (Fully Connected Layer)

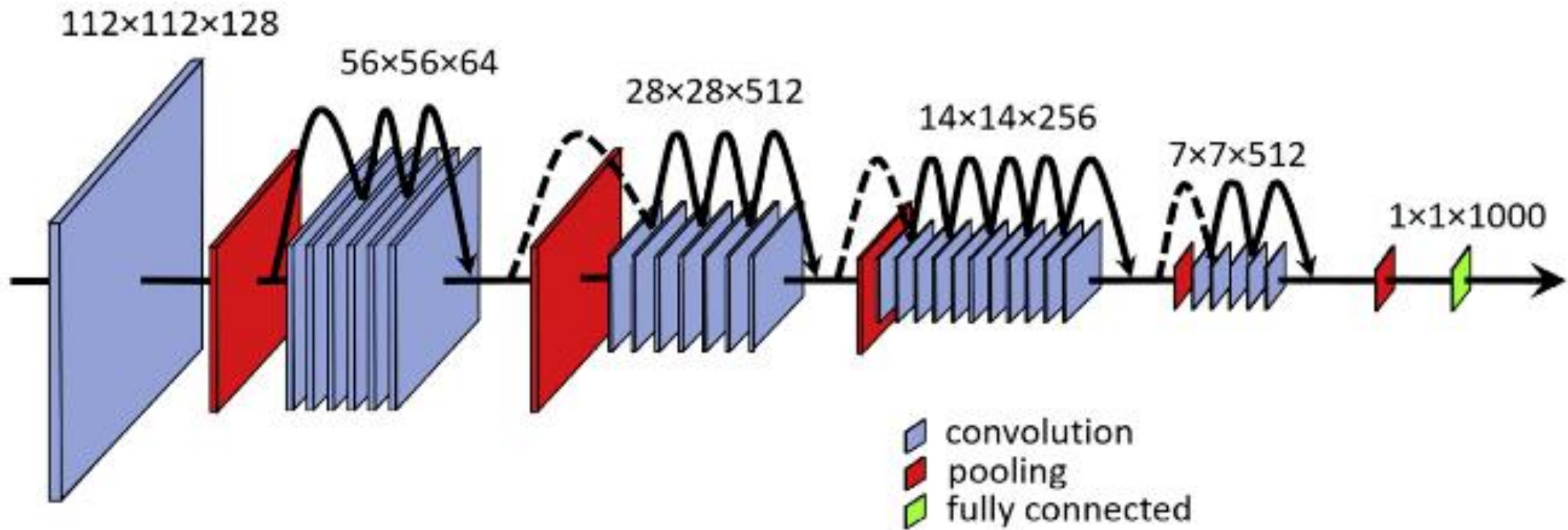
- The Fully Connected layer is a traditional Multi Layer Perceptron that uses a softmax activation function in the output layer.
- The purpose of the Fully Connected layer is to use these features for classifying the input image into various classes based on the training dataset.
- For example, the image classification task we set out to perform has four possible outputs.



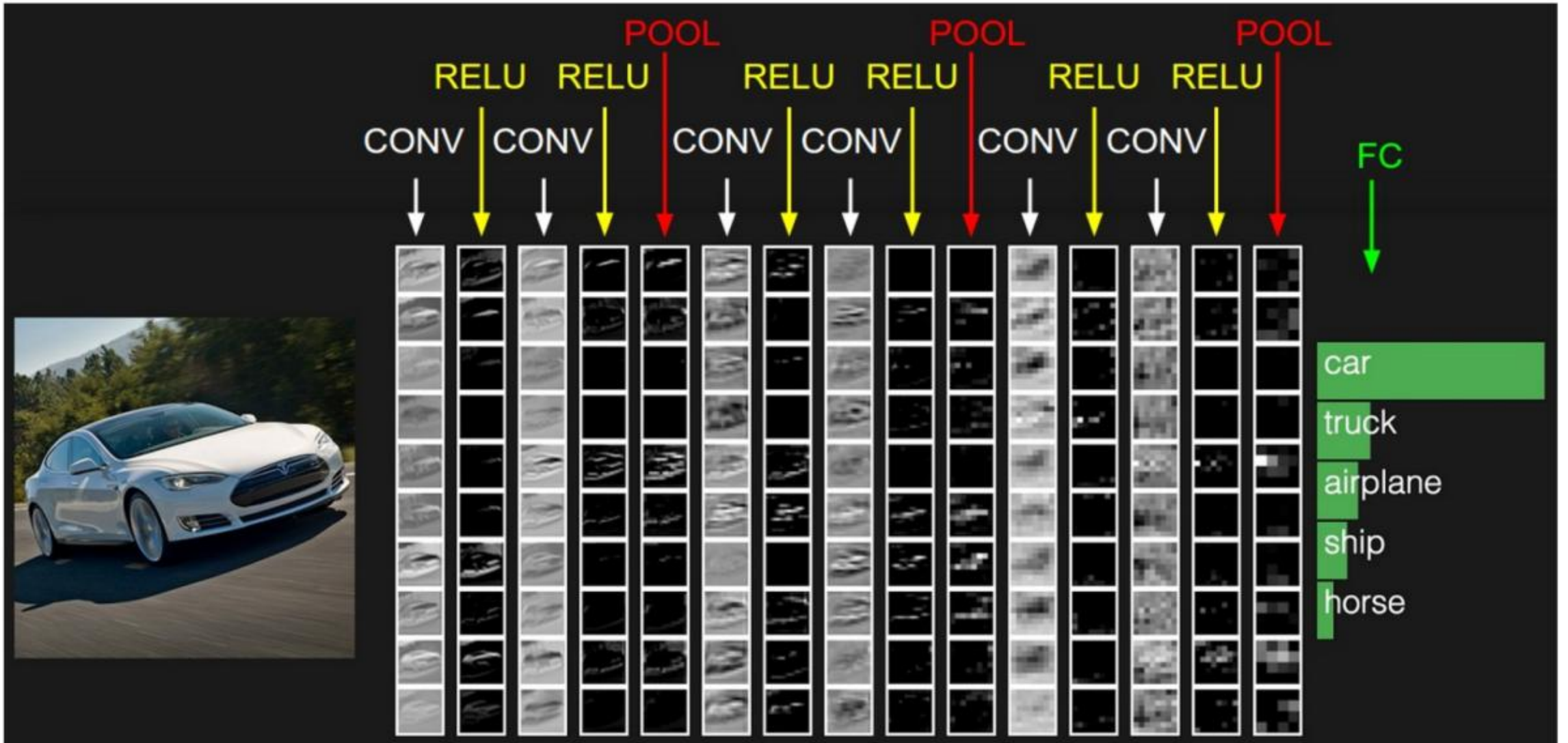
VGG Architecture



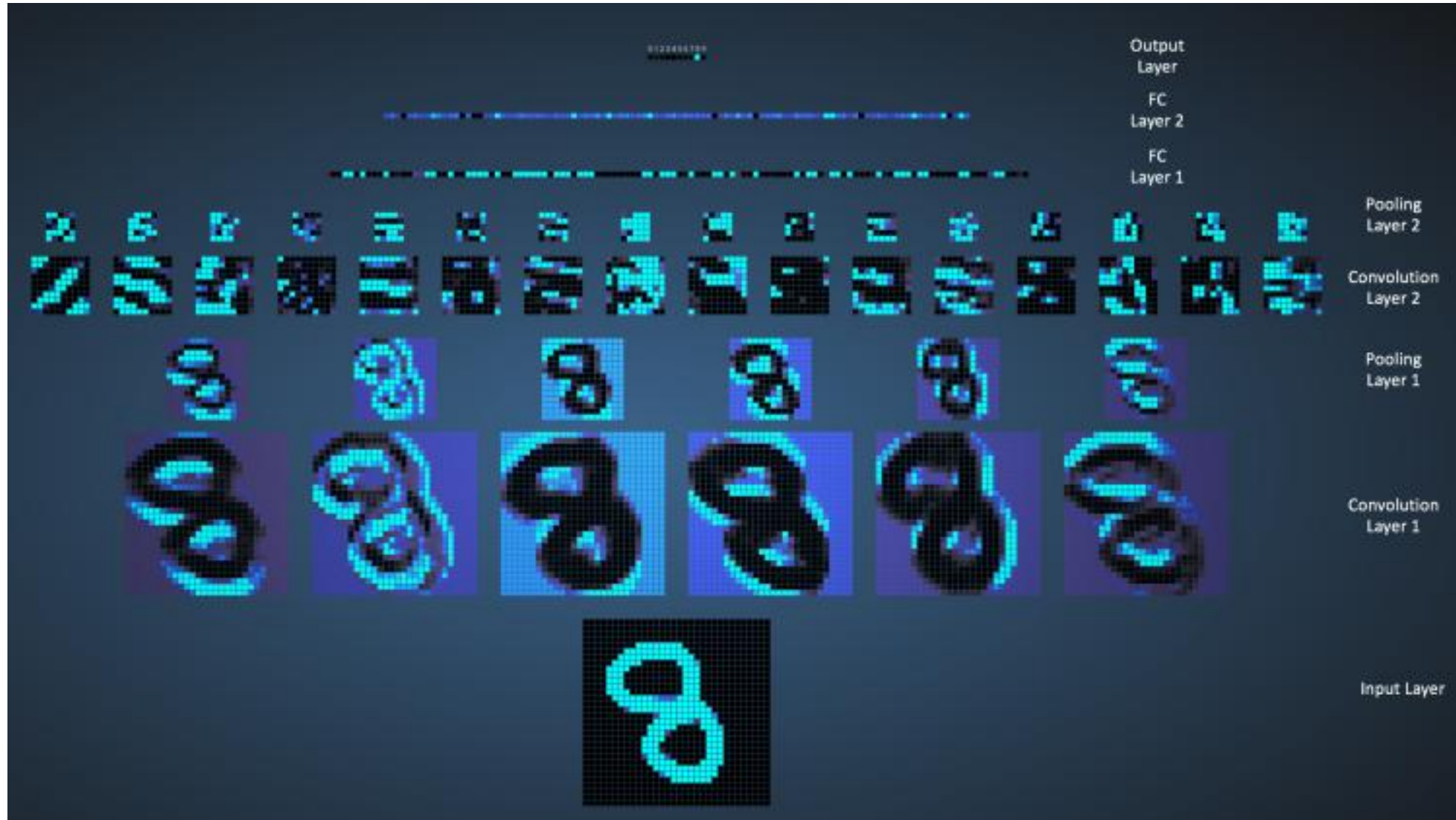
Resnet Architecture



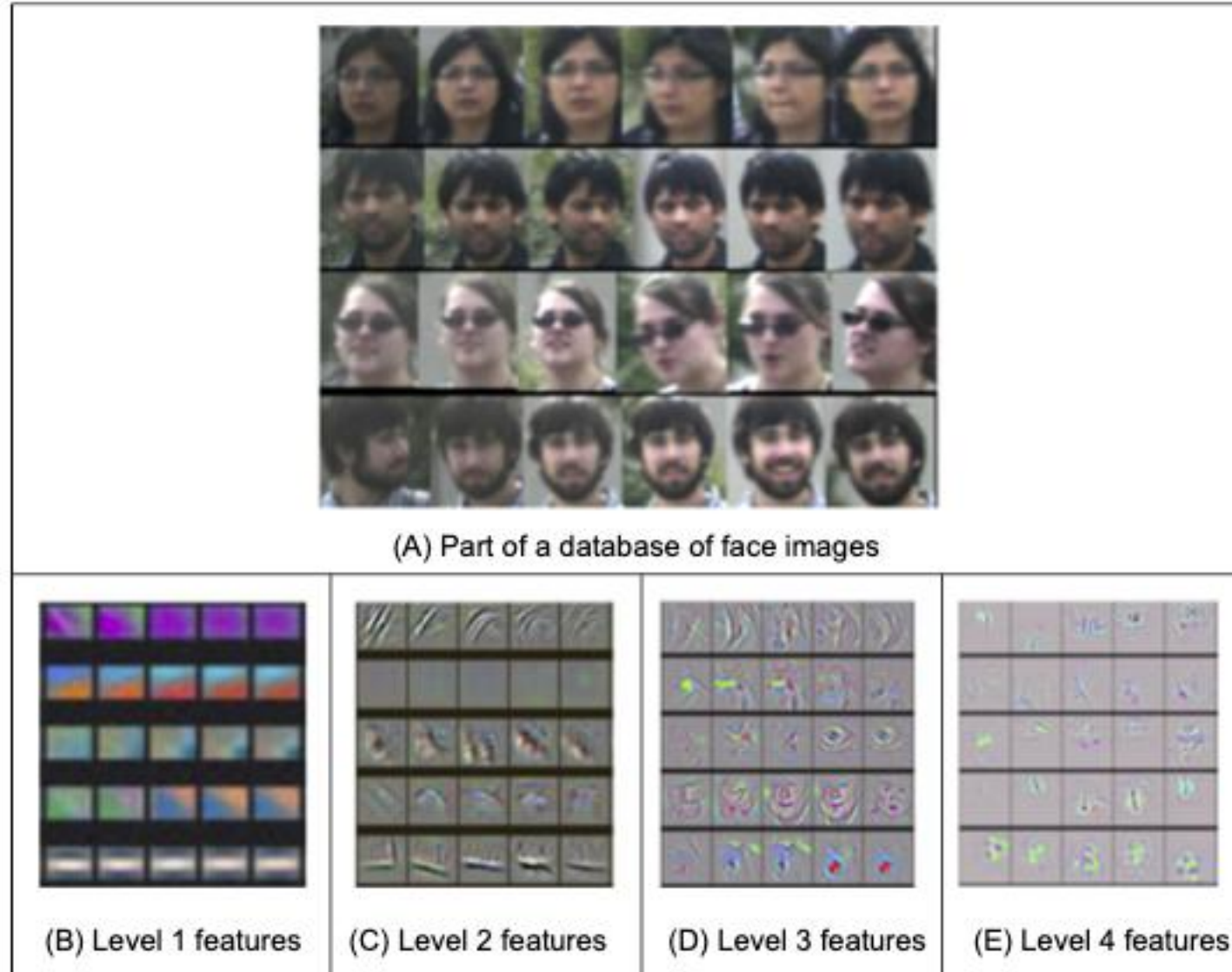
Visualizing Convolutional Neural Networks



Visualizing Convolutional Neural Networks



Visualizing Convolutional Neural Networks



Example : Bird Classifier

data set :

CIFAR10 data set contains 6,000 pictures of birds and 52,000 pictures of things that are not birds.

Caltech-UCSD Birds-200-2011 data set that has another 12,000 bird pics.

bird images



Example : Bird Classifier

data set :

CIFAR10 data set contains 6,000 pictures of birds and 52,000 pictures of things that are not birds.

Caltech-UCSD Birds-200-2011 data set that has another 12,000 bird pics.

non-bird images



Example : Bird Classifier

Some of the birds that our network correctly identified as birds. Let's call these **True Positives**:



Some images that our network correctly identified as “not a bird”. These are called **True Negatives**:



Example : Bird Classifier

Some images that we thought were birds but were not really birds at all. These are our **False Positives**:



Some images of birds that we didn't correctly recognize as birds. These are our **False Negatives**:



Example : Bird Classifier

The validation set of 15,000 images, here's how many times the predictions fell into each category:

Results for 15,000 Validation Images

(6000 images are birds, 9000 images are not birds)

	Predicted 'bird'		Predicted 'not a bird'
Bird	5,450 <i>True Positives</i>	TP	FN 550 <i>False Negatives</i>
Not a Bird	162 <i>False Positives</i>	FP	TN 8,838 <i>True Negatives</i>

Precision <i>If we predicted 'bird', how often was it really a bird?</i>	97.11% <i>(True Positives ÷ All Positive Guesses)</i>
Recall <i>What percentage of the actual birds did we find?</i>	90.83% <i>(True Positives ÷ Total Birds in Dataset)</i>

$$precision = \frac{TP}{TP + FP}$$

$$recall = \frac{TP}{TP + FN}$$

ML.NET

- An open source and cross-platform machine learning framework for .NET Application



Sentiment analysis

Analyze the sentiment of customer reviews using a binary classification algorithm.



Product recommendation

Recommend products based on purchase history using a matrix factorization algorithm.



Price prediction

Predict taxi fares based on parameters such as distance traveled using a regression algorithm.



Customer segmentation

Identify groups of customers with similar profiles using a clustering algorithm.



Object detection

Recognize objects in an image using an ONNX deep learning model.



Fraud detection

Detect fraudulent credit card transactions using a binary classification algorithm.



Sales spike detection

Detect spikes and changes in product sales using an anomaly detection model.



Image classification

Classify images (for example, broccoli vs. pizza) using a TensorFlow deep learning model.



Sales forecasting

Forecast future sales for products using a regression algorithm.

ML.NET

There are several ML scenarios that are supported by the ML.NET CLI:

- **Classification** - Use this when you want to predict which category data belongs in (for example, analyzing sentiment of customer reviews as either positive or negative).
- **Image classification** - Use this when you want to predict which category an **image** belongs to (for example, predicting if an image is of a cat or a dog).
- **Regression (for example, value prediction)** - Use this when you want to predict a numeric value (for example, predicting house price).
- **Recommendation** - Use this when you want to recommend items to users based on historical ratings (for example, product recommendation).

ML.Net Model Builder

An easy to understand visual interface to build, train, and deploy custom machine learning models. Prior machine learning expertise is not required.

The screenshot displays the Visual Studio Workload Installer interface. At the top, there are four tabs: 'Workloads', 'Individual components', 'Language packs', and 'Installation locations'. The 'Workloads' tab is active, showing two categories: 'Web & Cloud (4)' and 'Desktop & Mobile (5)'. In the 'Desktop & Mobile (5)' section, the '.NET desktop development' workload is selected, indicated by a blue checkmark and a red rectangular highlight. To the right, the 'Installation details' panel shows a list of components. Under the '.NET desktop development' section, several components are listed, with 'ML.NET Model Builder' highlighted by a red rectangular box. At the bottom, the 'Location' is set to 'C:\Program Files\Microsoft Visual Studio\2022\Preview', and the 'Total space required' is 5.84 GB. The 'Install while downloading' and 'Install' buttons are visible at the bottom right.

Workloads Individual components Language packs Installation locations

Web & Cloud (4)

- ASP.NET and web development
Build web applications using ASP.NET Core, ASP.NET, HTML/JavaScript, and Containers including Docker supp...
- Azure development
Azure SDKs, tools, and projects for developing cloud apps and creating resources using .NET and .NET Framework....
- Python development
Editing, debugging, interactive development and source control for Python.
- Node.js development
Build scalable network applications using Node.js, an asynchronous event-driven JavaScript runtime.

Desktop & Mobile (5)

- .NET Multi-platform App UI development
Build Android, iOS, Windows, and Mac apps from a single codebase using C# with .NET MAUI.
- .NET desktop development**
Build WPF, Windows Forms, and console applications using C#, Visual Basic, and F# with .NET and .NET Frame...
- Desktop development with C++
Build modern C++ apps for Windows using tools of your...
- Windows application development
Build applications for the Windows platform using WinUI

Installation details

- ▼ .NET desktop development
 - ▼ Included
 - ✓ .NET desktop development tools
 - ✓ .NET Framework 4.7.2 development tools
 - ✓ C# and Visual Basic
 - ▼ Optional
 - ✓ Development tools for .NET
 - ✓ .NET Framework 4.8 development tools
 - ✓ Entity Framework 6 tools
 - ✓ .NET profiling tools
 - ✓ IntelliCode
 - ✓ Just-In-Time debugger
 - ✓ Live Share
 - ✓ ML.NET Model Builder**
 - ✓ GitHub Copilot
 - ✓ Blend for Visual Studio
 - ☐ F# desktop language support
 - ☐ PreEmptive Protection - Dotfuscator
 - ☐ .NET Framework 4.6.2-4.7.1 development t...

Location
C:\Program Files\Microsoft Visual Studio\2022\Preview [Change...](#)

By continuing, you agree to the [license](#) for the Visual Studio edition you selected. We also offer the ability to download other software with Visual Studio. This software is licensed separately, as set out in the [3rd Party Notices](#) or in its accompanying license. By continuing, you also agree to those licenses.

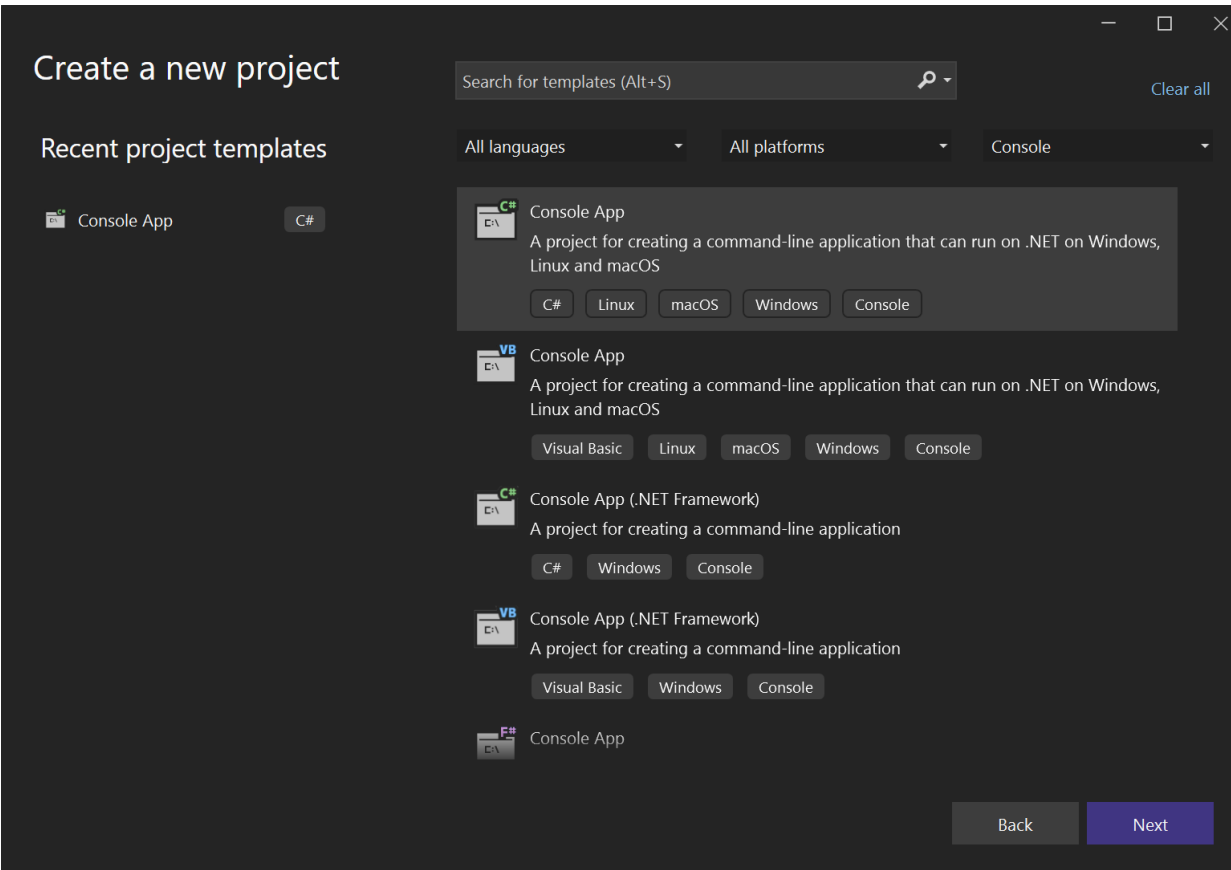
Total space required 5.84 GB

[Remove out-of-support components](#)

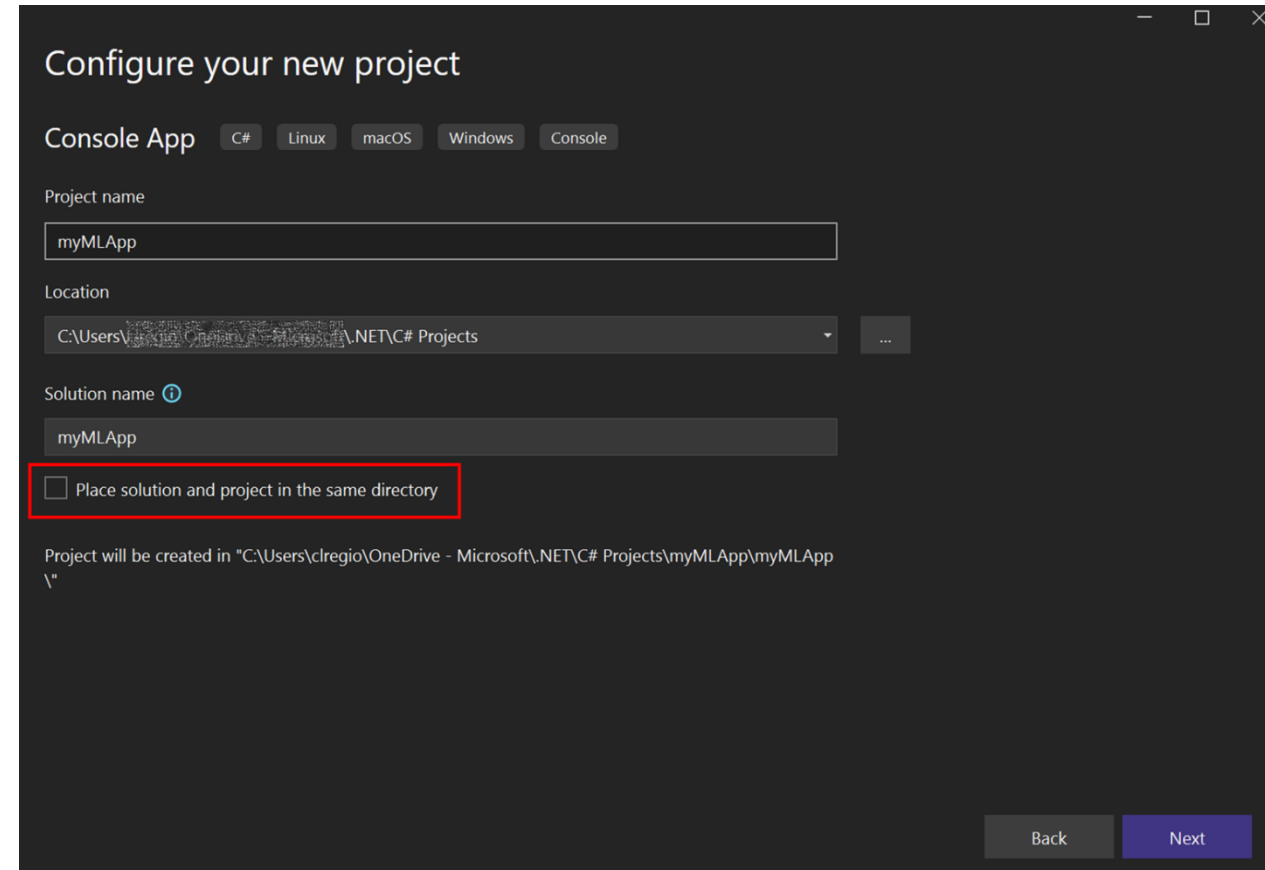
[Install while downloading](#) [Install](#)

Create an App

1. Select **Create a new project** from the Visual Studio 2022 start window.
2. Select the **C# Console App** project template.



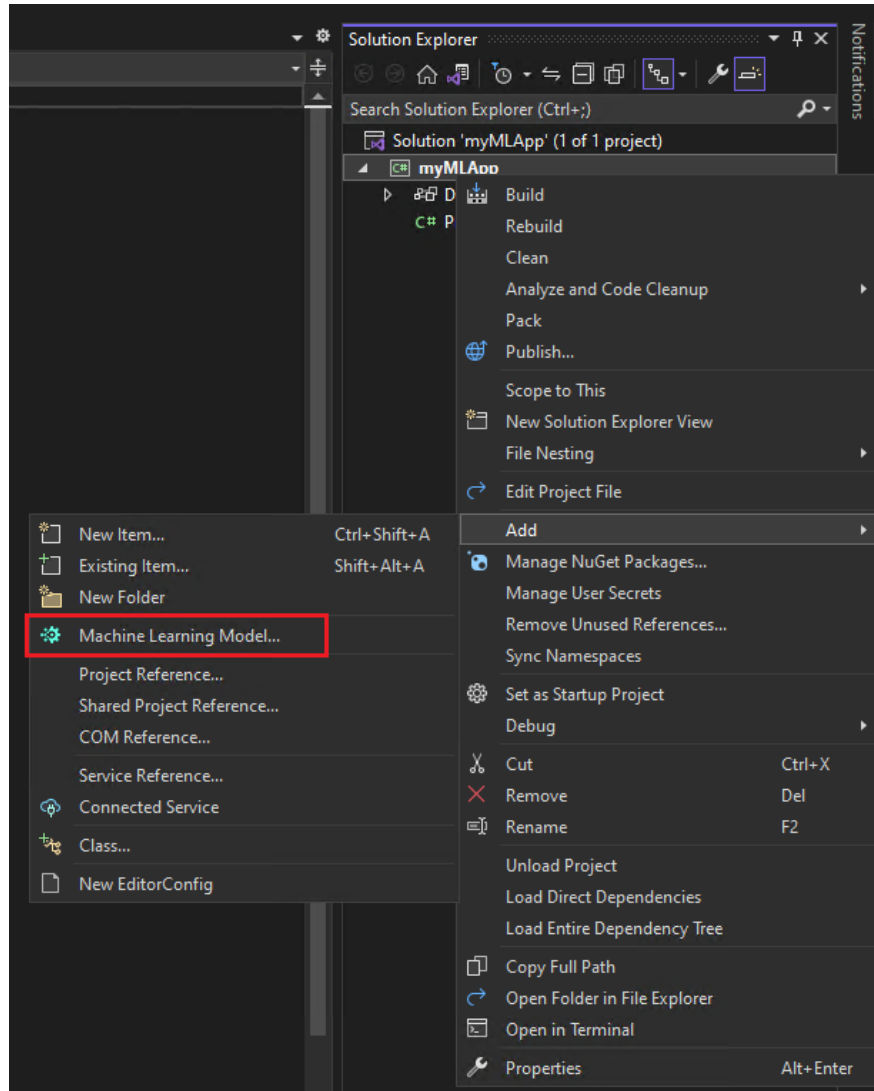
3. Change the project name to myMLApp.
4. Make sure **Place solution and project in the same directory** is unchecked.



5. Select the **Next** button.
6. Select **.NET 8.0 (Long Term support)** as the Framework.
7. Select the **Create** button. Visual Studio creates your project and loads the Program.cs file.

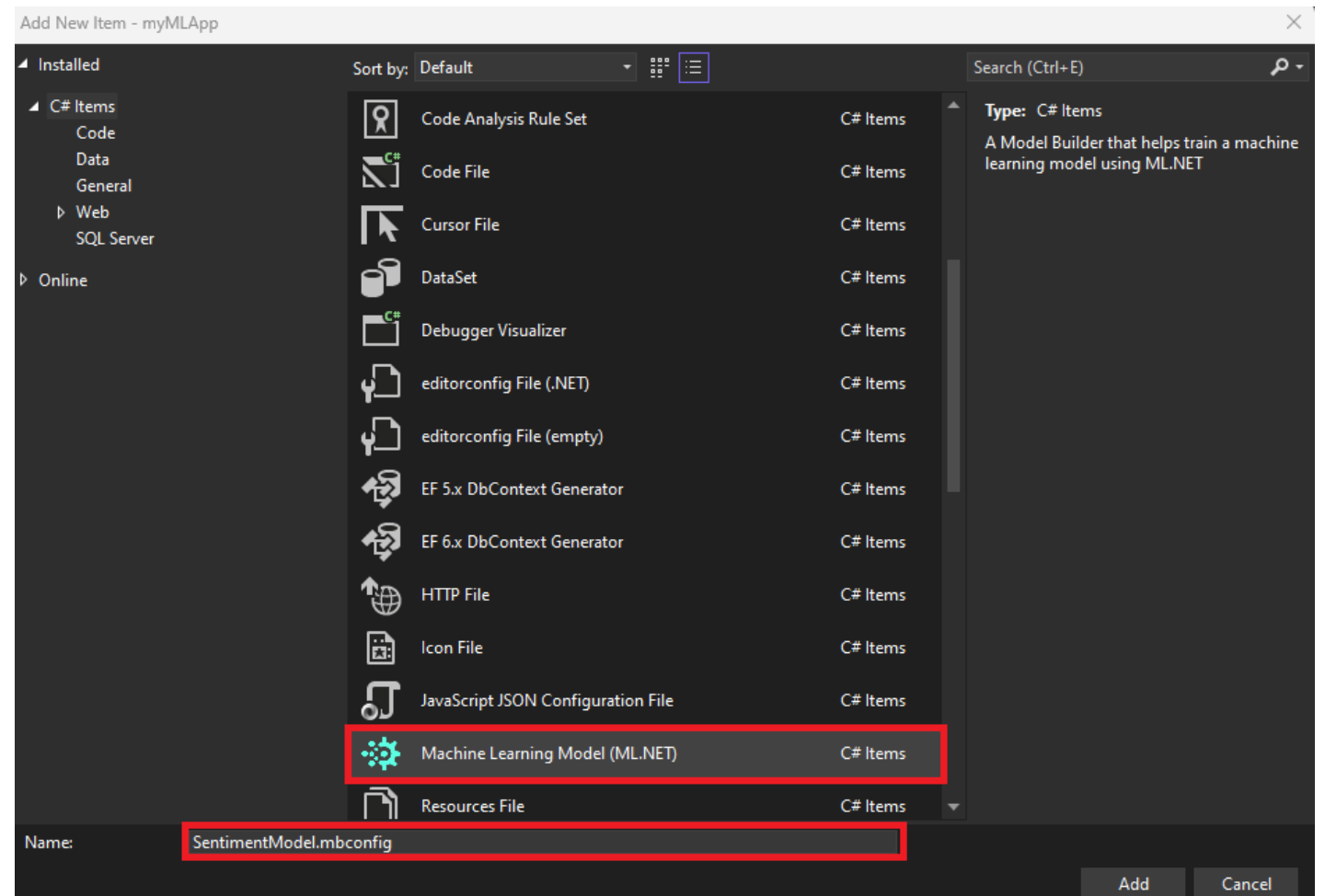
Add machine learning

1. Right-click on the myMLApp project in **Solution Explorer** and select **Add > Machine Learning Model**.

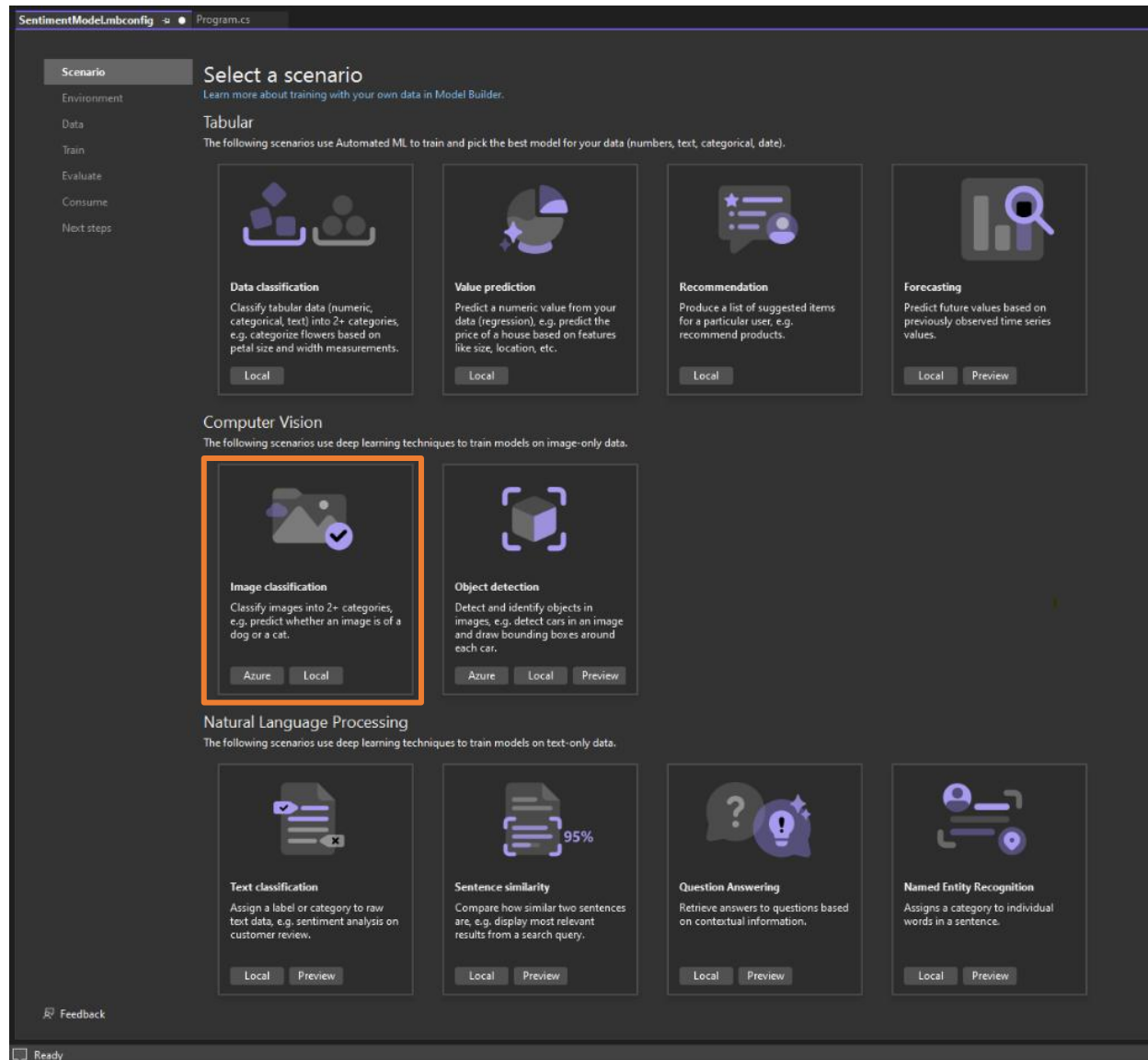


2. In the **Add New Item** dialog, make sure **Machine Learning Model (ML.NET)** is selected.

3. Change the **Name** field to **ImageClassificationModel.mbconfig** and select the **Add** button.

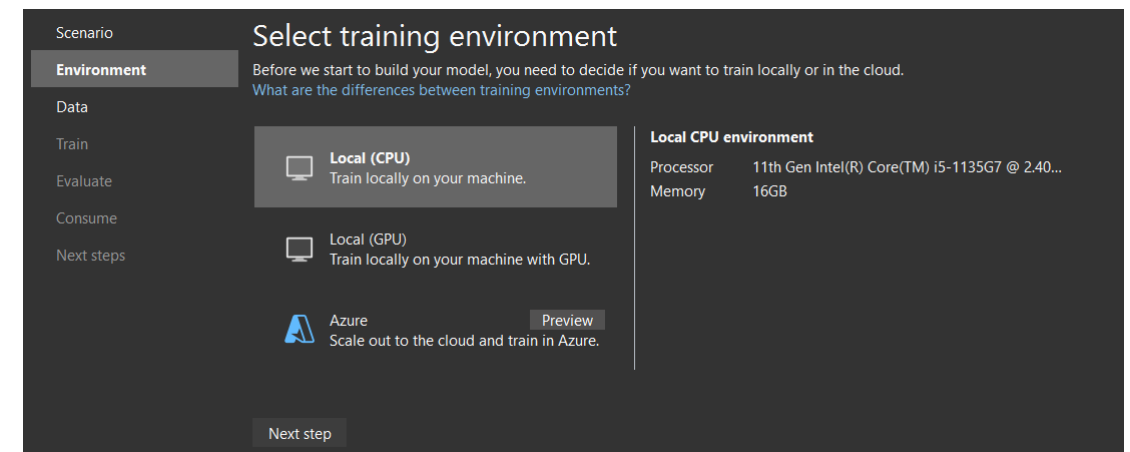


Pick a scenario



Task	Scenario
Binary classification	Data classification
Multiclass classification	Data classification
Image classification	Image classification
Text classification	Text classification
Regression	Value prediction
Recommendation	Recommendation
Forecasting	Forecasting

1. In the Model Builder Scenario screen, select the **Data classification** scenario, since you're predicting which category a comment falls into (positive or negative).
2. After selecting the **Image classification** scenario, you must choose your training environment. While some scenarios support training in Azure, Classification currently only supports local training, so keep the **Local** environment selected and move on to the **Data** step.



Download Fruits 360 Dataset

- Fruits 360 Dataset at <https://www.kaggle.com/moltean/fruits>
- Four classes are selected: Braeburn apple, Meyer lemon, mango, and raspberry.



Image size: 100x100 pixels

Train: Braeburn apple (492), Meyer lemon (490), mango (490), and raspberry (490)

Test: Braeburn apple (164), Meyer lemon (166), mango (166), and raspberry (166)

Add Data and Train

Scenario

Environment

Data

Train

Evaluate

Consume

Next steps

Add data

In order to build a model, you must add image data.
[How do I get sample datasets and learn more?](#)

Input

Select the folder which contains all your images. This folder should organize your photos into separate labeled sub-folders.

Select a folder:

Supported file formats: .png, .jpg, .jpeg, .gif.

[Advanced data options...](#)

Data Preview

Total images 1962. Showing 8/492.

train:

Apple Bra... (492)

Lemon Me... (490)

Mango (490)

Raspberry (490)



Next step

Example folder structure:

- Sample Folder
 - Sub Folder Label 1
 - Image 1
 - Image 2
 - Sub Folder Label 2
 - Sub Folder Label 3

Scenario

Environment

Data

Train

Evaluate

Consume

Next steps

Train

Training setup summary

[Advanced training options...](#)

Start training

Next step

Training Results

The screenshot shows the 'Train' tab in the Azure Machine Learning Studio interface. On the left is a sidebar with navigation links: Scenario, Environment, Data, Train (highlighted), Evaluate, Consume, and Next steps. The main area is titled 'Train' and contains a 'Training setup summary' dropdown, a link to 'Advanced training options...', a 'Train again' button, a green checkmark icon, and the text 'Training complete'. Below this is the 'Training results' section, which lists the following metrics:

- Best MicroAccuracy: 1.0000
- Best model: ImageClassificationMulti
- Training time: 328.22 seconds
- Models explored (total): 1
- Generated code-behind: ImageClassificationMLModel.consumption.cs, ImageClassificationMLModel.training.cs

At the bottom of the results section is a 'Next step' button.

- **Best MacroAccuracy** - This shows you the accuracy of the best model that Model Builder found. Higher accuracy means the model predicted more correctly on test data.
- **Best model** - This shows you which algorithm performed the best during Model Builder's exploration.
- **Training time** - This shows you the total amount of time that was spent training / exploring models.
- **Models explored (total)** - This shows you the total number of models explored by Model Builder in the given amount of time.
- **Generated code-behind** - This shows you the names of the files generated to help consume the model or train a new model.

Evaluate your model

Scenario

Environment

Data

Train

Evaluate

Consume

Next steps

Evaluate

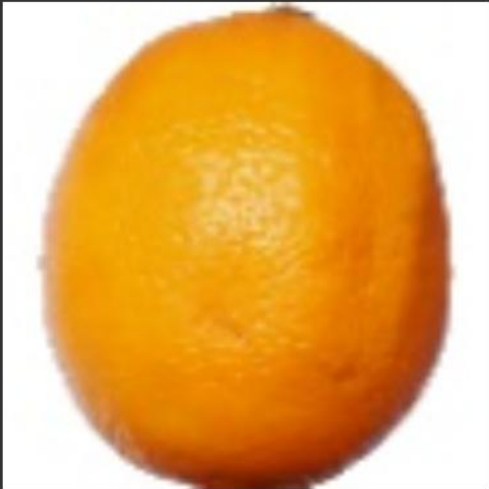
Results of training for your model can be found below.
[How do I understand my model performance?](#)

Best model:

MicroAccuracy: 1.0000

Model: ImageClassificationMulti

Try your model



[Try another image](#)

Results

- Lemon Meyer 100%
- Apple Braeburn < 1%
- Raspberry < 1%
- Mango < 1%

 Feedback

Generate code

Scenario

Environment

Data

Train

Evaluate

Consume

Next steps

Consume

Your model is ready to use!

Use the code below in an end-user application to consume your model.

Code snippet:

Project templates

These projects use the model most recently trained on 26/2/2568 at 13:06.

Console app
A .NET console application that uses your model to make predictions.

Add console app to solution

Web API
An ASP.NET Core web API that consumes your model.

Add web app to solution

×

Add sample console app

It will use the most recent model created on 26/2/2568 at 13:06.

Project name

ImageClassificationMLModel_ConsoleApp1

Add to solution

Cancel

Solution Explorer

Search Solution Explorer (Ctrl+;)

Solution 'myMLApp' (2 of 2 projects)

C#

ImageClassificationMLModel_ConsoleApp1

- Dependencies
- ImageClassificationMLModel.mbconfig
- Program.cs

C#

myMLApp

- Dependencies
- ImageClassificationMLModel.mbconfig
- Program.cs

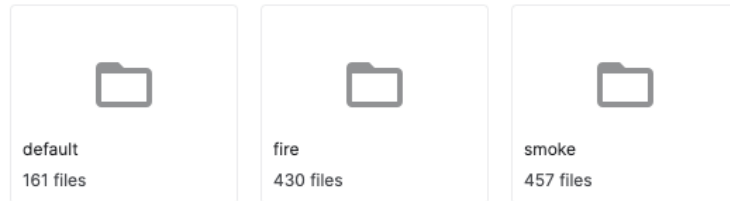
```
Microsoft Visual Studio Debug Console
To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.
2025-02-26 13:18:03.455397: I tensorflow/compiler/xla/service/service.cc:168] XLA service 0x26019ee8f50 initialized for platform Host (this does not guarantee that XLA will be used). Devices:
2025-02-26 13:18:03.455519: I tensorflow/compiler/xla/service/service.cc:176] StreamExecutor device (0): Host, Default Version
Class      Score
-----
Apple Braeburn      0.9876797
Mango              0.011257786
Lemon Meyer        0.0009192974
Raspberry          0.00014315051

C:\Users\Suriya\source\repos\myMLApp\ImageClassificationMLModel_ConsoleApp1\bin\Debug\net8.0\ImageClassificationMLModel_ConsoleAp
pl.exe (process 21924) exited with code 0 (0x0).
Press any key to close this window . . .
```

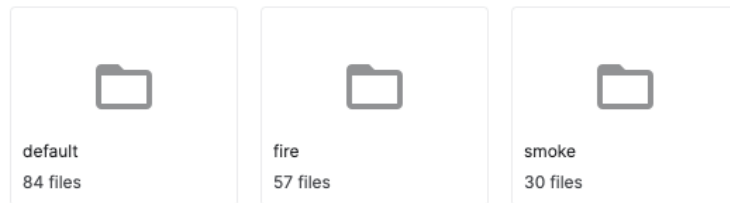

Smoke Fire Dataset

- Fire and Smoke dataset <https://www.kaggle.com/ashutosh69/fire-and-smoke-dataset>

< train (3 directories)



< test (3 directories)



Smoke Fire Dataset

- ✓ 1. Scenario
- ✓ 2. Environment
- ✓ 3. Data
- ✓ 4. Train
- 5. Evaluate
- 6. Code

Train

Model Builder automatically sets the training time for image classification based on the size of your dataset.

[How long should I train for?](#)

Training setup summary 

Train again



Training complete

Training results

Best accuracy: 88.17%
Best model: DNN + ResNet50
Training time: 1144.29 seconds
Models explored (total): 1

Next step: evaluate your model

Evaluate

Case Study #4 : ML.Net

- ให้ลองเล่น ML.Net โดยศึกษาวิธีการใช้งานได้ที่
<https://dotnet.microsoft.com/en-us/learn/ml-dotnet/get-started-tutorial/intro>
- โดยเลือกใช้งานแบบ image classification with ML.NET
- สามารถใช้รูปภาพจาก <https://www.kaggle.com> ได้
- ส่งผลการใช้งานได้ที่ LEB2